

Opdracht Softwareontwerp 2

ONTWERPVERANTWOORDING

Daphné Trachez – Groep B

Endless Flyer in MonoGame

Voor de eindevaluatie van Softwareontwerp 2 koos ik om de opdracht Endless Flyer in Monogame uit te werken. Mijn focus lag voornamelijk op het opleveren van een functioneel spel, maar die op een flexibele en herbruikbare manier ontworpen is. Daarnaast vond ik het ook belangrijk om de code zo leesbaar en overzichtelijk mogelijk te houden.

Het project is opgebouwd rondom de centrale Game-klasse (standaard aangemaakt als de Game1-klasse), waarin de grafische functionaliteiten via de GraphicsDeviceManager worden beheerd. Het oproepen van de SpriteBatch, die instaat voor het renderen en visueel aanpassen van 2D-afbeeldingen en lettertypes via de Draw-methode. Maar ook het opstarten van het eigenlijke spel, het laden van de content en het spelverloop (via de Update-methode) worden hier geregeld.

Deze verantwoordelijkheden heb ik verder verdeeld over meerdere klassen die op hun beurt elk een specifieke deeltaak uitvoeren. De architectuur is als volgt opgezet:

In het Core-domein (kern componenten van de applicatie) bracht ik klassen als de AssetsManager, twee Exception klassen (AssetsException en GraphicsException) en twee facade klassen (GraphicsFacade en KeyboardFacade) respectievelijk onder in Assets, Exceptions en Facades.

De AssetsManager staat in voor het inladen van de content in het spel. Het krijgt de opdracht om alle fysieke bestanden (zoals .png afbeeldingen en .spritefont lettertypes) vanuit de Content-map in het werkgeheugen te laden. Hiervoor heb ik twee methodes GetTexture en GetFont die helpen de content op te halen via de ingebouwde ContentManager. Wanneer dat lukt, bewaart hij deze als variabelen in een dictionary, waardoor een link wordt gelegd tussen de naam en de afbeelding of het lettertype. Andere klassen kunnen zo via de AssetsManager deze texturen en fonts opvragen, zonder deze telkens zelf te moeten inladen. Wanneer het niet lukt om de content op te vragen, wordt een AssetNotInitializedException gegooid.

De AssetNotInitializedException klasse gooit een foutmelding wanneer de specifieke asset niet kan worden gevonden (een innerException). De GraphicsFacadeNotInitialized-

Exception gooit daarentegen een gewone exception: ofwel laden de graphics, ofwel laden ze niet.

Om de afhandeling van de gebruikersinvoer en de schermconfiguratie te vereenvoudigen, heb ik gebruik gemaakt van het facade-patroon. Deze specifieke klassen heb ik onder één Facades-folder gebracht, om de projectstructuur eenvoudig te houden. Zo dienen deze klassen als een vereenvoudigde brug naar het framework: beide Facade-klassen zijn bewust static gemaakt, zodat ze overal in het project opvraagbaar zijn. Hierdoor hoeven deze klassen niet telkens geïnitieerd te worden en hoeven ze niet via constructors aan elke onderliggende klasse te worden meegegeven. Dit houdt de code overzichtelijk en direct toegankelijk voor componenten die schermdimensies of toetsaanslagen nodig hebben.

Om te kunnen navigeren tussen de verschillende schermen en fases van het spel (menu, spelen, pauze, game-over en topscores) heb ik het state-patroon toegepast. Er is een abstracte basisklasse `AbstractState` waaraan de `GameContext` wordt meegegeven. Specifieke speltoestanden zoals `MenuState`, `PlayState`, `PauseState`, `GameOverState` en `TopScoreState` erven hiervan over. Deze abstracte basisklasse fungeert als een blauwdruk en dwingt af dat elk scherm een gestandaardiseerde `Update`- en `Draw`-methode bevat. Elke status is zelf verantwoordelijk voor zijn eigen logica en rendering. Zo kan de `PauseState` bijvoorbeeld de actie bevriezen door simpelweg de `Update` van de `PlayState` niet aan te roepen, maar wel diens `Draw` aan te roepen zodat het scherm zichtbaar blijft. In de `MenuState` luistert de `Update` enkel naar de toetsen 1, 2 en 3 om het spel te starten en in de `Update` van de `PlayState` wordt gecontroleerd of `_currentMode.IsGameOver` waar is. Zodra dat zo is, regelt deze state zelf de overgang naar de `GameOverState`.

Hoewel elke status autonoom beslist wanneer er een overgang plaatsvindt, gebeurt de daadwerkelijke wissel via de `GameContext` (door middel van `Context.ChangeState(...)`). De `GameContext` fungeert hierbij als een soort statusbeheerder. Het houdt bij welke status actief is en stuurt de rendering en update-logica hier dynamisch naar door. Door de `GameContext` mee te geven aan de constructor van de `AbstractState`, krijgen alle schermen direct toegang tot essentiële gedeelde componenten, zoals de reeds besproken `AssetsManager`, de `ScoreRepository` (voor de LiteDB databank) en de verschillende `Factories`.

Binnen de `Environment`-folder bevindt zich naast de `GameContext` ook de afhandeling van de verschillende spelmodi. In plaats van de specifieke logica voor één of twee vliegtuigen rechtstreeks in de `PlayState` te programmeren, heb ik dit gedrag geabstraheerd achter de

IGameMode interface. De PlayState hoeft hierdoor alleen maar de Update en Draw van de meegegeven modus aan te roepen, zonder te hoeven weten of het een singleplayer- of doubleplayer-sessie is.

Omdat veel functionaliteit in het spel identiek is voor elke modus, heb ik de AbstractGameMode ontworpen. Deze abstracte klasse neemt gedeelde logica omtrent allerhande spelonderdelen op zich, waaronder het tekenen van de wereld, het beheren van alle afzonderlijke entiteiten en een methode voor wanneer het vliegtuig botst. Door deze gemeenschappelijke fundering hoeven de concrete implementaties — SinglePlayerMode en DoublePlayerMode — zich uitsluitend te focussen op wat hen uniek maakt.

Tijdens het uitschrijven van deze klasse botste ik echter op een probleem: de klasse werd te groot en kreeg te veel verantwoordelijkheden. Na overweging heb ik deze abstracte klasse verder afgesplitst naar een aparte Background klasse, waar alle achtergrond-gerelateerde code werd in afgezonderd (zoals o.a. het opvragen van de textuur via de AssetsManager, het opvragen van de snelheid via de GameSettings en het updaten van de verticale positie voor het scrollend effect). Daarnaast heb ik het introduceren van alle karakters in de spelwereld overgebracht naar een aparte klasse: de IntroducingCharacters klasse.

Het spawnen, updaten en bijhouden van alle speelbare en niet-speelbare objecten - vliegtuig(en), vijanden, bomen en huizen - zijn overgeleverd naar deze gespecialiseerde manager-klasse. In deze klasse heb ik ook nog de Blockades- en InvadersSprites samengevoegd in één gezamenlijke lijst Threats om de botsingsdetectie eenvoudiger te maken in de AbstractGameMode. Door deze refactoring is de verantwoordelijkheid van de AbstractGameMode gereduceerd tot het coördineren van het spelgebeuren. Het communiceert met de Background en IntroducingCharacters klasse zonder belast te worden met de interne werking ervan.

Voor het ontwerpen van deze objecten zelf heb ik gebruik gemaakt van het sprite-patroon. De drie klassen Blockade-, Invader- en PlayerSprite erven over van de basisklasse Sprite en zijn ondergebracht in de Objects-folder. De basisklasse bevat een aantal gedeelde eigenschappen: texture, position, speed en scale. Maar ook herbruikbare logica zoals UpdateHorizontalPosition, UpdateVerticalPosition en de Draw-methode, die één keer gedefinieerd worden. Verder is een belangrijk onderdeel van de Sprite klasse de Collision-methode. Deze berekent een 'onzichtbare rechthoek' op basis van de huidige positie, de textuurdimensies en de schaalfactor. Hierdoor heeft elke entiteit een accuraat botsingsgebied.

Om de weergave van deze entiteiten zo leesbaar mogelijk te coderen heb ik gebruik gemaakt van Extension-Methods. In de Extensions-folder bevindt zich de SpriteBatchExtensions klasse. Om te voorkomen dat ik de parameters overal opnieuw moest uitschrijven, heb ik deze verpakt in beknopte extensies. Hierdoor kan ik in de Draw-methode van de Sprite simpelweg `spriteBatch.Draw(Texture, Position, Scale)` aanroepen, waarna de extensie de extra parameters vanzelf aanvult met standaardwaarden, zoals bijvoorbeeld `Color.White`.

Naast het optimaliseren van deze gedeelde tekenlogica, was het ook cruciaal om het specifieke gedrag van elke afzonderlijke entiteit efficiënt te structureren. De 'invader' (het vijandelijk vliegtuig ofwel 'Angry Plane') maakt gebruik van een bewegingsstrategie die via de constructor in de `InvaderSprite` wordt meegegeven. Hiervoor hanteerde ik een strategy-patroon om de beweging los te koppelen van het object dat de beweging uitvoert. In de `AngryPlaneMovementStrategies`-folder staat centraal de interface `IMovementStrategy` die een methode opstelt voor het bepalen van de beweging (`DetermineInvaderMovement`). Deze methode neemt de huidige positie, afmetingen en snelheid van de vijand, evenals de verstreken tijd en geeft een `Vector2` terug die de nieuwe beweging voorstelt. Op basis van deze interface heb ik twee concrete strategieën uitgewerkt: De `VerticalMovementStrategy` die een neerwaartse beweging berekend, gebaseerd op de snelheid en de verstreken tijd en de `HorizontalMovementStrategy` die een richtingsvariabele (`_horizontalDirection`) bijhoudt. Zodra het vliegtuig de linker- of rechterkant van het venster raakt, wordt deze richting omgedraaid. Tegelijkertijd berekent het een lichte daling zodat het vliegtuig met de achtergrond meebeweegt.

Deze vijandelijke objecten (de invaders en de blockades) komen op willekeurige tijdstippen tevoorschijn in de spelwereld. Om dit tijdbeheer overzichtelijk te houden, heb ik de spawner-logica apart gehouden in specifieke klassen onder de `Spawners`-folder. De abstracte klasse `BaseSpawner` neemt de verantwoordelijkheid voor het bijhouden van de tijd op zich. Het telt de verstreken milliseconden op en vergelijkt deze met een berekend doel-interval. Pas wanneer de timer dit interval overschrijdt, retourneert de `Update`-methode 'true' en wordt de timer gereset. De afgeleide klassen, zoals `HouseSpawner`, `TreeSpawner` en `AngryPlaneSpawner`, hoeven zich hierdoor enkel nog bezig te houden met hun specifieke timings. Via de abstracte methode `DetermineSpawnInterval` genereren zij een willekeurig tijdsinterval. Hierbij heb ik bewust vermeden om hardgecodeerde data te gebruiken: de minimale en maximale spawntijden worden opgevraagd uit de `GameSettings` klasse.

De spawners zijn in dit ontwerp aldus uitsluitend verantwoordelijk voor de timing: zij hebben geen weet van het creatieproces, dit valt buiten hun scope. De `IntroducingCharacters` klasse fungeert hierbij als de aanspreekbare manager. Deze klasse verifieert via de spawner of de timer is verstreken (`if (_houseSpawner.Update(gameTime))`) en stuurt bij een positief antwoord een verzoek naar de specifieke Factory om het object te bouwen.

Omdat er voortdurend nieuwe spelobjecten – zoals huizen, bomen en vijandelijke vliegtuigen – in de wereld gegenereerd moeten worden, heb ik ook het Factory patroon uitgewerkt. Dit patroon centraliseert specifiek de logica voor het aanmaken van de objecten. Om dubbele code tussen de verschillende fabrieken te voorkomen, erven ze allemaal over van de abstracte `BaseFactory`. Deze basisklasse levert essentiële, gedeelde functionaliteiten: een referentie naar de `GameContext` (om gemakkelijk bij de `AssetsManager` te kunnen), een gedeclareerd `Random` object en een herbruikbare hulpmethode (`CreateHorizontalPosition`) om willekeurige startcoördinaten op de X-as te berekenen. Waarom ik enkel voor een willekeurige horizontale startpositie geopteerd heb en niet voor een willekeurige verticale startpositie heeft te maken met het spelverloop. Oorspronkelijk had ik dit wel gerandomiseerd, maar dan kreeg ik het probleem dat de speler buiten het scherm startte of dat de invaders plots in het begin al in het speelveld aanwezig waren, waardoor een botsing heel snel kon gebeuren. Om deze reden is de verticale startpositie bepaald per entiteit: de obstakels komen van bovenaf buiten het scherm naar beneden en de speler begint net boven de onderkant van het scherm. Verder heb ik voor zowel de `Blockade`- als de `InvaderFactory` twee private methodes voorzien zodat er willekeurig gekozen kan worden tussen de twee soorten texturen. Voor de `PlayerFactory` heb ik een private `CreatePlayer`-methode voorzien om van daaruit een `SinglePlayer` of `DoublePlayer` te creëren, welke nog een specifieke `PlayerInput` meekrijgen.

Uit de opdracht begreep ik dat de vliegtuigen in de `singleplayer`-modus op meerdere manieren bestuurd moesten kunnen worden (pijltoetsen of letters), terwijl deze toetsen in de `doubleplayer`-modus strikt verdeeld moesten worden over twee afzonderlijke spelers. Om de `PlayerSprite` niet te overladen met deze logica, heb ik de inputverwerking volledig losgekoppeld en de `IPlayerInputService` interface ontworpen. De `PlayerSprite` vraagt deze interface op via zijn constructor en roept enkel methodes aan, zoals bijvoorbeeld `_inputService.MoveUp()`, waarna hij zijn positie aanpast.

De eigenlijke mapping van de toetsen gebeurt dan in de `PlayerInputService`. Via de statische methodes in deze klasse worden de specifieke toetsensets klaargezet: `ButtonsetOne` koppelt de acties aan de pijltoetsen, `ButtonsetTwo` koppelt de acties aan

de lettertoetsen (ZSQD) en de CombinedSet combineert de beiden. Deze laatste set heb ik via een extra PlayerInputService _extraSet kunnen creëren, om vervolgens deze mee te geven aan de singleplayer-modus. Hierdoor is de code een stuk beknopter geworden, waar ik oorspronkelijk drie afzonderlijke klassen had voorzien, heb ik de functionaliteit kunnen samenvoegen in één klasse.

Nu ik de meeste patronen en onderdelen van mijn applicatie heb besproken, rest er nog een belangrijk onderdeel. Want wat is een spel eigenlijk zonder het bijhouden van een score? Om de hoogste scores per spelmodus te kunnen bewaren in een LiteDB databank, heb ik gebruik gemaakt van het repository patroon. Dit patroon scheidt de data-gerelateerde logica van alle andere code. De basis van deze opzet is de IScoreRepository interface. Deze definieert methodes zoals SaveScore, ShowTopScores en DeterminePlayerRank. De verschillende spelstatussen, zoals de GameOverState en de TopScoreState, communiceren via deze interface (die centraal toegankelijk is via de GameContext). Hierdoor hebben de game states geen enkele weet van de databank: ze vragen enkel de data op of geven een score door.

De effectieve interactie met de LiteDB databank vindt uitsluitend plaats in de ScoreRepository klasse. Dit centraliseert alle database-bewerkingen. Een belangrijk aspect hiervan is het beheren van de database-grootte: de opdracht vereiste dat we per gamemodus enkel de top 5 bijhouden en dat overbodige data verwijderd moest worden. De RemoveScore-methode wordt automatisch na elke nieuwe score-toevoeging aangeroepen. Deze methode leest de scores uit, sorteert ze dalend, behoudt de beste vijf en wist de rest uit de database. Deze topscores worden dan op het einde van ieder spel weergegeven in de GameOverState volgens de gespeelde playermodus (single of double).

Nog een kleine toevoeging:

Op de meeste plaatsen haal ik hardgecodeerde waarden op via de GameSettings (onder de Identifiers-folder). Toch heb ik dit op een aantal plaatsen bewust niet gedaan omwille van de leesbaarheid van de code. Bijvoorbeeld in de BaseSpawner staat de timer terug op 0 wanneer het doel-interval wordt bereikt. Dit leest gemakkelijker dan dat je een woord zou moeten interpreteren – je ziet in een oogopslag dat de timer terug op 0 staat. Ook bij de berekening van de verticale startpositie (in de PlayerFactory) heb ik dit geschreven: GraphicsFacade.GetWindowHeight() - 100f, de speler start 100 pixels boven de onderkant van het scherm. Voor de omvang van dit spel vond ik het niet nodig om deze enkele getallen te gaan hardcoden in de GameSettings.